FTE4312: Blockchain and Decentralized Applications

Homework Assignment

Instructions: Answer in English. Show calculations for Part C. Keep explanations concise.
Note: Use the simplified rules stated in each question (toy models).

Part A — True / False (10 questions)

Write **T** or **F** for each statement, and **briefly explain your reasoning**.

1. SPV nodes validate PoW on headers and can request Merkle path proofs. _____
   Explanation: _____
2. In Bitcoin, miners prefer transactions with higher fee-per-byte when block space is limited. _____
   Explanation: _____
3. Bitcoin retargets difficulty every 2,016 blocks (a "period"). _____
   Explanation: _____
4. In Ethereum, EOAs have contract code stored by default. _____
   Explanation: _____
5. In Ethereum, storage is persistent, while memory is temporary for execution. _____
   Explanation: _____
6. If a call has leftover gas, the unused gas is refunded (i.e., not charged). _____
   Explanation: _____
7. Under EIP-1559, the base fee is burned and the priority fee is a tip. _____
   Explanation: _____
8. A transaction with an old nonce can still be executed again if it is re-broadcast. _____
   Explanation: _____
9. Locktime can delay when a Bitcoin transaction becomes valid for mining/relay. _____
   Explanation: _____
10. A Merkle root changes if any included transaction changes. _____
    Explanation: _____

Part B — Multiple Choice (10 questions)

Choose **ONE** answer (A–D) for each question, and **briefly explain your reasoning**.

1. If a transaction has inputs 0.900 BTC and outputs 0.895 BTC, the fee is:

   A. 0.005 BTC
   B. 0.0005 BTC
   C. 0.05 BTC
   D. 0.00005 BTC

   Explanation: _____

2. "Fee-per-byte" means:

   A. fee ÷ number of inputs
   B. fee ÷ transaction size in bytes
   C. fee × transaction size in bytes
   D. fee ÷ block size limit

   Explanation: _____

3. Which is a correct statement about Bloom filters?

   A. They are exact-match databases
   B. They are probabilistic search filters
   C. They prevent PoW mining
   D. They replace Merkle trees

   Explanation: _____

4. In EIP-1559, miners/validators are incentivized mainly by:

   A. Base fee
   B. Priority fee
   C. Burn fee
   D. Nonce

   Explanation: _____

5. Which Ethereum field prevents replay attacks?

   A. Recipient
   B. Nonce
   C. Data
   D. Block number

   Explanation: _____

6. If a transaction runs out of gas:

   A. Effects kept; gas refunded
   B. Effects discarded; gas not refunded

C. Effects discarded; gas fully refunded

D. Effects partially applied

Explanation: _____

7. A Merkle proof size (in hashes) for a tree with 2^k leaves is:

A. 2^k

B. k

C. k^2

D. $2k$

Explanation: _____

8. In Bitcoin, the block header is roughly:

A. 8 bytes

B. 80 bytes

C. 800 bytes

D. 1 MB

Explanation: _____

9. In the UTXO model, balance of an address is computed as:

A. last received UTXO only

B. sum of all UTXOs spendable by that address

C. sum of all outputs ever received (spent or not)

D. sum of all inputs ever used

Explanation: _____

10. A transaction with MaxFeePerGas $<$ BaseFee will:

A. be included immediately

B. not be includable until BaseFee drops

C. execute for free

D. burn the priority fee only

Explanation: _____

Part C — Computational Problems (4 problems)

C1) Fee ladder with locktime

Background on Locktime (nLockTime): In Bitcoin, each transaction can include a *locktime* field. If locktime is set to a block height H, the transaction cannot be included in any block until the blockchain reaches height H. This allows creating "time-delayed" transactions. If multiple conflicting transactions (spending the same UTXO) exist, only one can ultimately be confirmed. By creating several versions with different locktimes and fees, a sender can implement a "fee escalation" strategy: if the low-fee version isn't confirmed quickly, a higher-fee version becomes valid later.

Alice has **one UTXO worth 0.2023 BTC** and wants to pay Bob within **3 hours**. Assume 1 block $\approx$ 10 minutes, so 3 hours $\approx$ **18 blocks**.

Alice will create **10 conflicting transactions** t_0 to t_9, all spending the same input, but with increasing fees and later locktimes.

- Maximum fee she is willing to pay: $\delta = 0.0010$ BTC
- Use this design:
  - timelock values: $T + 0, T + 2, T + 4, \ldots, T + 18$
  - fee values: $0.0001, 0.0002, \ldots, 0.0010$ BTC
  - output to Bob in each transaction $= 0.2023 -$ fee

Task: Fill in for each t_i:

Transaction	Input (BTC)	Output to Bob (BTC)	Timelock
t_0	0.2023	_____	$T+$ _____
t_1	0.2023	_____	$T+$ _____
t_2	0.2023	_____	$T+$ _____
t_3	0.2023	_____	$T+$ _____
t_4	0.2023	_____	$T+$ _____
t_5	0.2023	_____	$T+$ _____
t_6	0.2023	_____	$T+$ _____
t_7	0.2023	_____	$T+$ _____
t_8	0.2023	_____	$T+$ _____
t_9	0.2023	_____	$T+$ _____

C2) Miner selection by fee-per-byte

A block can include at most **900 bytes** of transactions (toy model). Miners choose transactions by **descending fee/byte** until the block is full.

Tx	Size (bytes)	Fee (satoshis)
A	400	8000
B	250	3000
C	600	15000
D	200	2800
E	500	6000
F	100	900

Tasks:

1. Compute each transaction's fee-per-byte.
2. List which transactions get included.
3. Compute total miner fee revenue.

C3) Difficulty retarget

Bitcoin retarget rule (simplified):

$$\text{newDifficulty} = \text{oldDifficulty} \times \frac{\text{expectedTime}}{\text{elapsedTime}}$$

Suppose:

- oldDifficulty = 2000
- expectedTime for a period = 20,160 minutes
- elapsedTime measured = 16,128 minutes

Task: Compute newDifficulty.

C4) UTXO set under a fork + reorg

You observe blocks arriving in this order: **B1** → **B2** → **B3** → **B4**.

- B2 extends B1.
- B3 forks from B1.

- B4 extends B3.
- The node switches to a new chain only if it is **strictly longer**.

Transactions:
B1: Tx1 spends $g_0 = 5.0$ BTC $\rightarrow$ creates

- $o_1 = 3.0$ BTC to Bob
- $o_2 = 1.9$ BTC to Alice

(so fee is implicit)
B2 (on top of B1): Tx2 spends $o_1 = 3.0$ BTC $\rightarrow$ creates

- $o_3 = 1.5$ BTC to Carol
- $o_4 = 1.4$ BTC to Bob

B3 (fork from B1): Tx3 spends $o_1 = 3.0$ BTC $\rightarrow$ creates

- $o_5 = 2.9$ BTC to Dave

B4 (on top of B3): Tx4 spends $o_5 = 2.9$ BTC $\rightarrow$ creates

- $o_6 = 2.0$ BTC to Erin
- $o_7 = 0.8$ BTC to Dave

Tasks:

1. What is the node's **active-chain UTXO set after B2**?
2. After receiving B4, which chain becomes active, and what is the **final UTXO set**?
3. For each output o_1–o_7, mark whether it **ever entered** the node's UTXO set and whether it is **in the final** UTXO set.

Part D — Bitcoin Difficulty Adjustment

Background: In Bitcoin, every node has its own clock. We assume that the clock skew of honest miners is at most 1 hour. Every block contains a timestamp specified by the miner who mined that block. A newly mined block is accepted only if its timestamp is strictly greater than the median timestamp of the previous 11 blocks, and less than *node_local_time* + 2 *hours* (timestamp bounding consensus rule).

The protocol tries to keep an expected block interval (time between two blocks) of 10 minutes. To achieve this, the mining difficulty parameter is adjusted every 2016 blocks ($\sim$ 2 weeks). Let's call this 2016 consecutive blocks a *period*. The difficulty adjustment is based on how long it took to mine the period. More precisely:

$$new_difficulty = old_difficulty \cdot \frac{expected_time}{elapsed_time}$$

where:

- $expected_time = 2016 \times 10 \times 60$ seconds
- $elapsed_time = max_timestamp(period) - min_timestamp(period)$

a) If more miners enter the Bitcoin ecosystem during a period, does the difficulty parameter for the next period increase or decrease?

b) If there was no upper bound on an accepted block's timestamp, how could an adversarial miner affect the difficulty parameter?

Assume an attacker manages to make timestamps in 4032 consecutive blocks look like this:

$$\underbrace{[t_0,\, t_0+1,\, t_0+2,\, \ldots,\, t_0+2014,\, \mathbf{t_1}}_{\text{2016 blocks (period}_1\text{)}},\, t_0+2016,\, t_0+2017,\, \ldots,\, t_0+4030,\, \mathbf{t_2}]_{\text{2016 blocks (period}_2\text{)}}$$

where:

- t_0 = time snapshot at some arbitrary moment
- $t_0 + 1 = t_0$ plus 1 second
- $t_1 = t_0 + (2016 \times 10 \times 60)$ seconds
- $t_2 = t_0 + 2 \times (2016 \times 10 \times 60)$ seconds

c) In this attack, what happens to the difficulty parameter at the end of $period_2$?